Assignment 2: Policy Gradient

Andrew ID: yuansu
Collaborators: Gemini (Guided Learning)
NOTE: Please do **NOT** change the sizes of the answer blocks or plots.

5 Small-Scale Experiments

5.1 Experiment 1 (Cartpole) – [5 points total]

5.1.1 Configurations

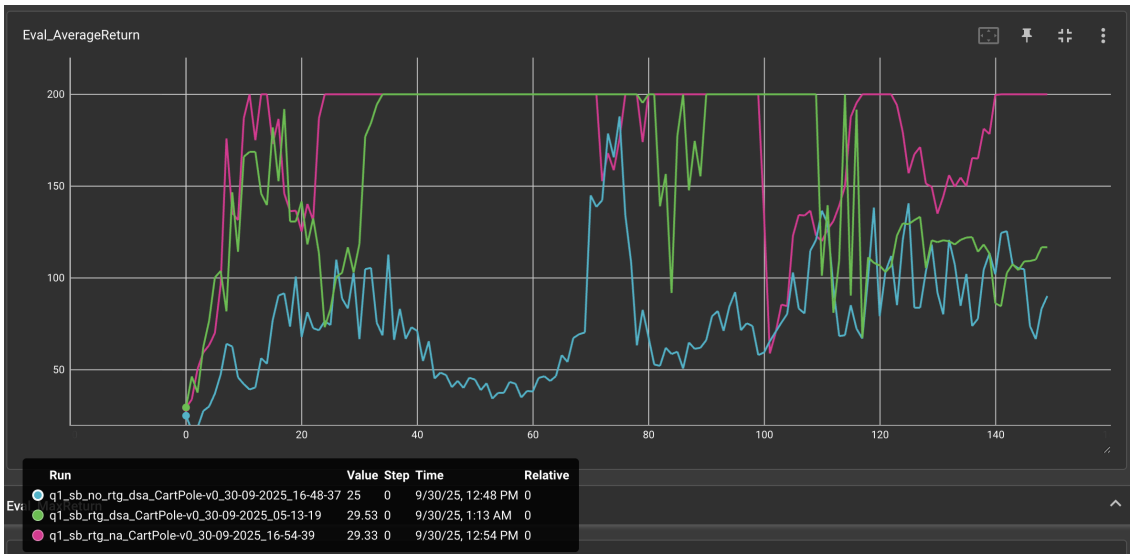
Q5.1.1

```
python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -dsa --exp_name q1_sb_no_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -rtg -dsa --exp_name q1_sb_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -rtg --exp_name q1_sb_rtg_na

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -dsa --exp_name q1_lb_no_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -rtg -dsa --exp_name q1_lb_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -rtg --exp_name q1_lb_rtg_na
```

5.1.2 Plots

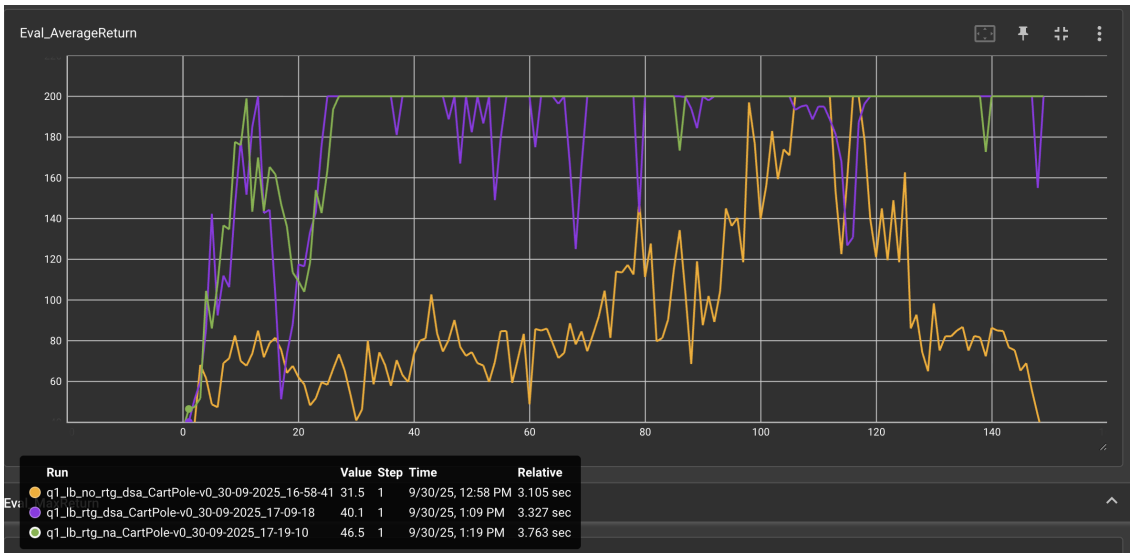
5.1.2.1 Small batch – [1 points]

Q5.1.2.1



5.1.2.2 Large batch – [1 points]

Q5.1.2.2



5.1.3 Analysis

5.1.3.1 Value estimator – [1 points]

Q5.1.3.1

In both configurations, experiments with reward-to-go perform significantly better than those without it. This is mainly because by only considering rewards from the current timestep onward, the variance can be much lower. The reduction is particularly evident when examining Eval_StdReturn metrics.

5.1.3.2 Advantage standardization – [1 points]

Q5.1.3.2

Further standardization is always preferred. In the small batch experiments, the pink curve, which includes standardization, converges and remain stable throughout most of the time. While the green curve, though also converges, regresses to a stochastic state toward the end of the rollouts.

5.1.3.3 Batch size – [1 points]

Q5.1.3.3

Larger batch size definitively helps reduce variance. In a small batch set up, gradients tend to be noisy, which poses challenge for model learning. Even after convergence, small batches impose high variance in gradient estimator, so the model is not 'finetuning' at it's final state but rather random updating, which explains the drop in performance in the end.

5.2 Experiment 2 (InvertedPendulum) – [4 points total]

5.2.1 Configurations – [1.5 points]

Q5.2.1

```
python rob831/scripts/run_hw2.py --env_name InvertedPendulum-v4 \
    --ep_len 1000 --discount 0.92 -n 100 -l 2 -s 64 -b 500 -lr 5e-2 -rtg \
    --exp_name q2_b500_r52
```

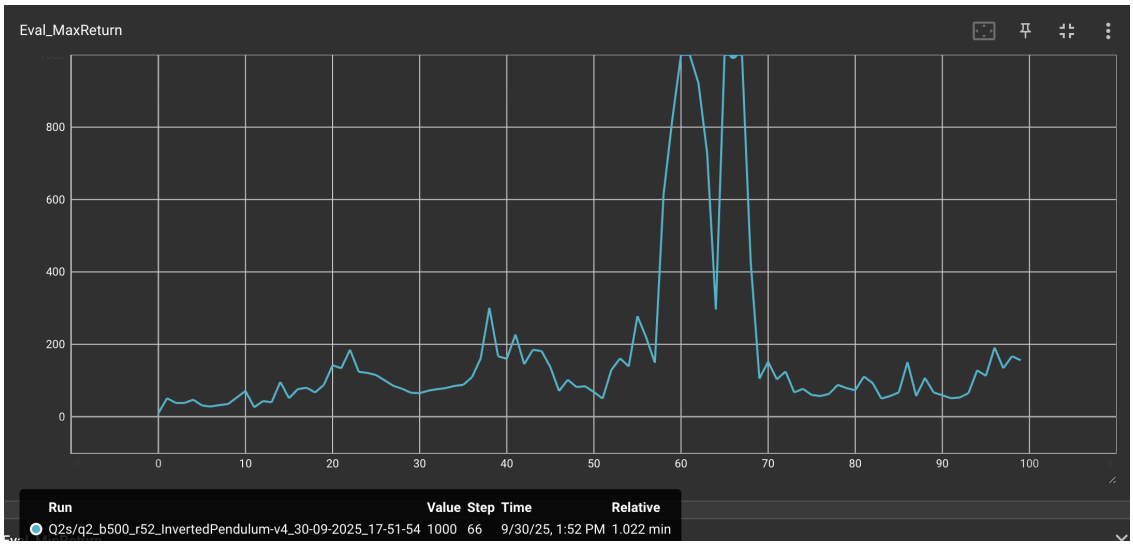
5.2.2 smallest b* and largest r* (same run) – [1.5 points]

Q5.2.2

b* = 500, r* = 0.05

5.2.3 Plot – [1 points]

Q5.2.3



7 More Complex Experiments

7.1 Experiment 3 (LunarLander) – [1 points total]

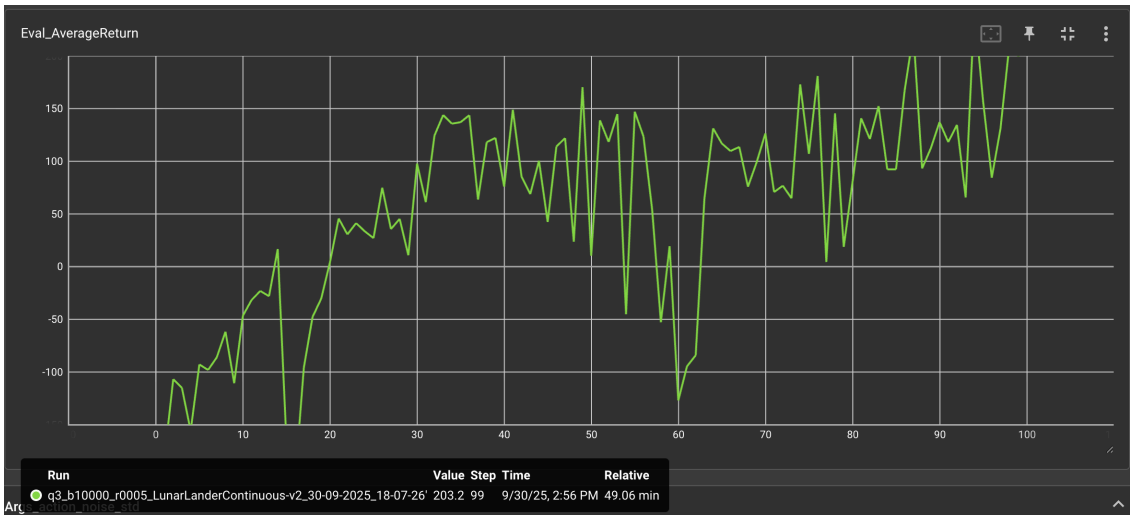
7.1.1 Configurations

Q7.1.1

```
python rob831/scripts/run_hw2.py \
    --env_name LunarLanderContinuous-v4 --ep_len 1000
    --discount 0.99 -n 100 -l 2 -s 64 -b 10000 -lr 0.005 \
    --reward_to_go --nn_baseline --exp_name q3_b10000_r0005
```

7.1.2 Plot – [1 points]

Q7.1.2



7.2 Experiment 4 (HalfCheetah) – [1 points]

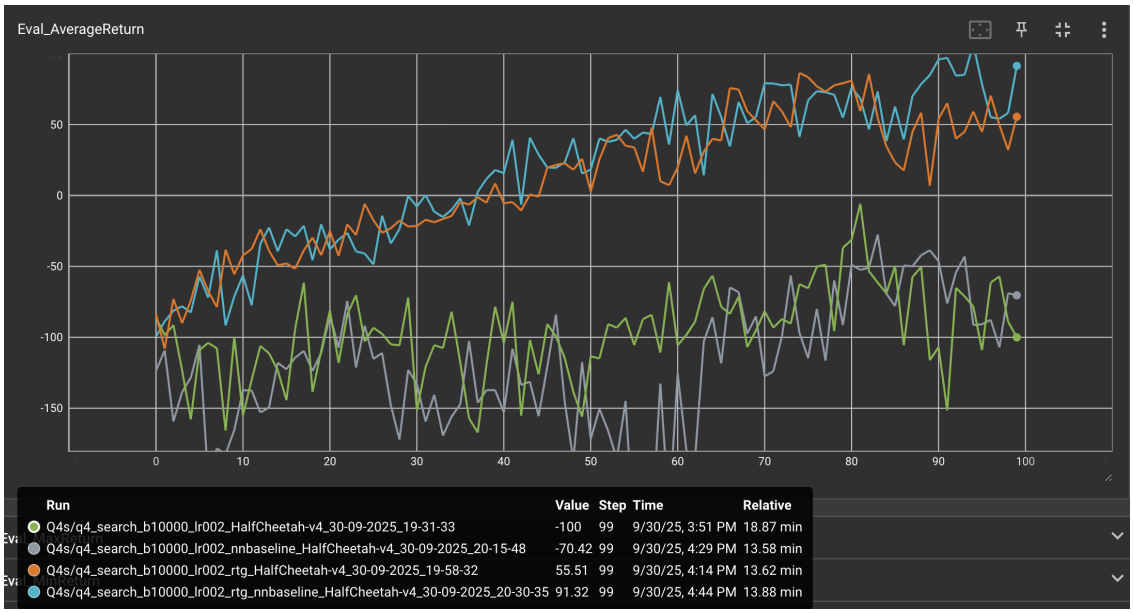
7.2.1 Configurations

Q7.2.1

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 \
    --exp_name q4_search_b10000_lr002
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 -rtg \
    --exp_name q4_search_b10000_lr002_rtg
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 --nn_baseline \
    --exp_name q4_search_b10000_lr002_nnbaseline
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b 10000 -lr 0.02 -rtg --nn_baseline \
    --exp_name q4_search_b10000_lr002_rtg_nnbaseline
```

7.2.2 Plot – [1 points]

Q7.2.2



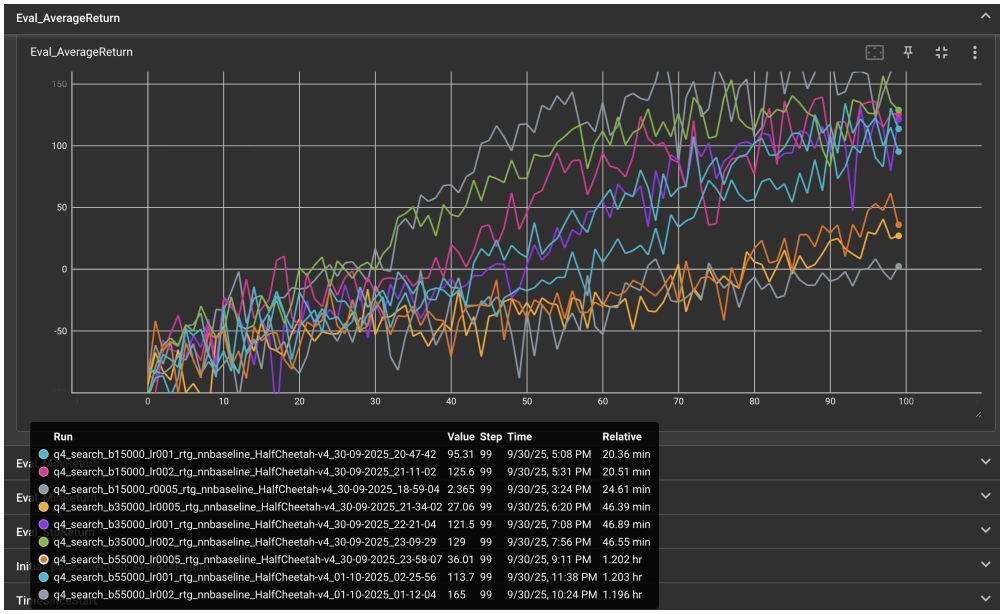
7.2.3 (bonus) Optimal b* and r* – [0.5 points]

Q7.2.3

b* = 10000, r* = 0.02

7.2.4 (bonus) Plot – [0.5 points]

Q7.2.4



7.2.5 (bonus) Describe how b* and r* affect task performance – [0.5 points]

Q7.2.5

Same as the previous findings, model tends to favor a larger batch size to reduce noise. However, pairing a large batch size with a small learning rate backfires as we are now learning way too slowly. Since a large batch size provides smaller variance and more stable updates, taking a small step here can be too conservative and inefficient. Generally the learning rate should scale proportionally with the batch size.

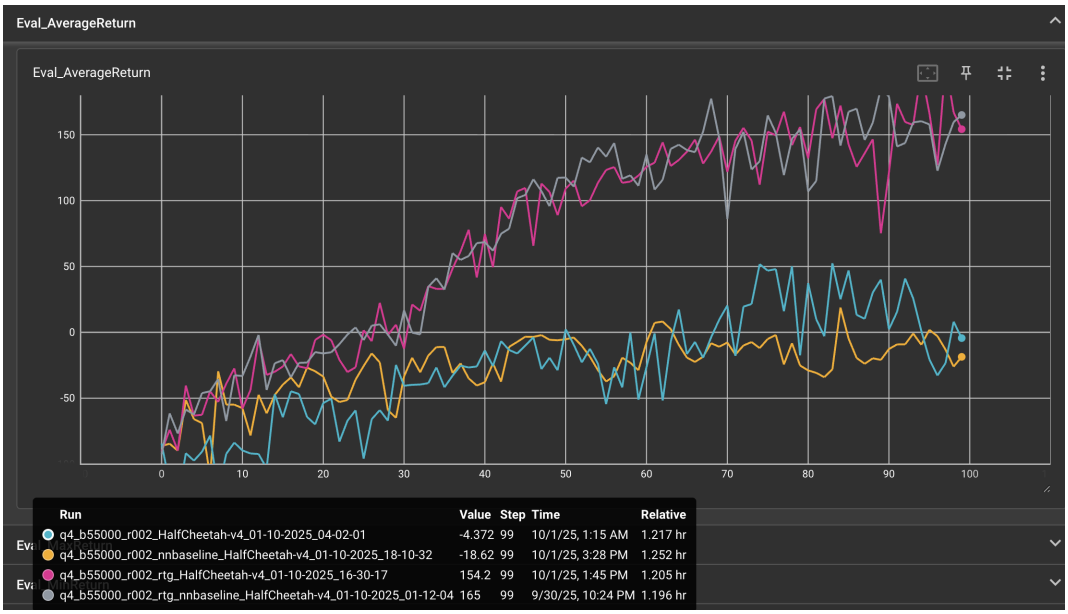
7.2.6 (bonus) Configurations with optimal b* and r* – [0.5 points]

Q7.2.6

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> \
    --exp_name q4_b<b*>_r<r*>

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> -rtg \
    --exp_name q4_b<b*>_r<r*>_rtg

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> --nn_baseline \
    --exp_name q4_b<b*>_r<r*>_nnbaseline

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> -rtg --nn_baseline \
    --exp_name q4_b<b*>_r<r*>_rtg_nnbaseline
```

7.2.7 (bonus) Plot for four runs with optimal b* and r* – [0.5 points]

Q7.2.7



8 Implementing Generalized Advantage Estimation

8.1 Experiment 5 (Hopper) – [4 points]

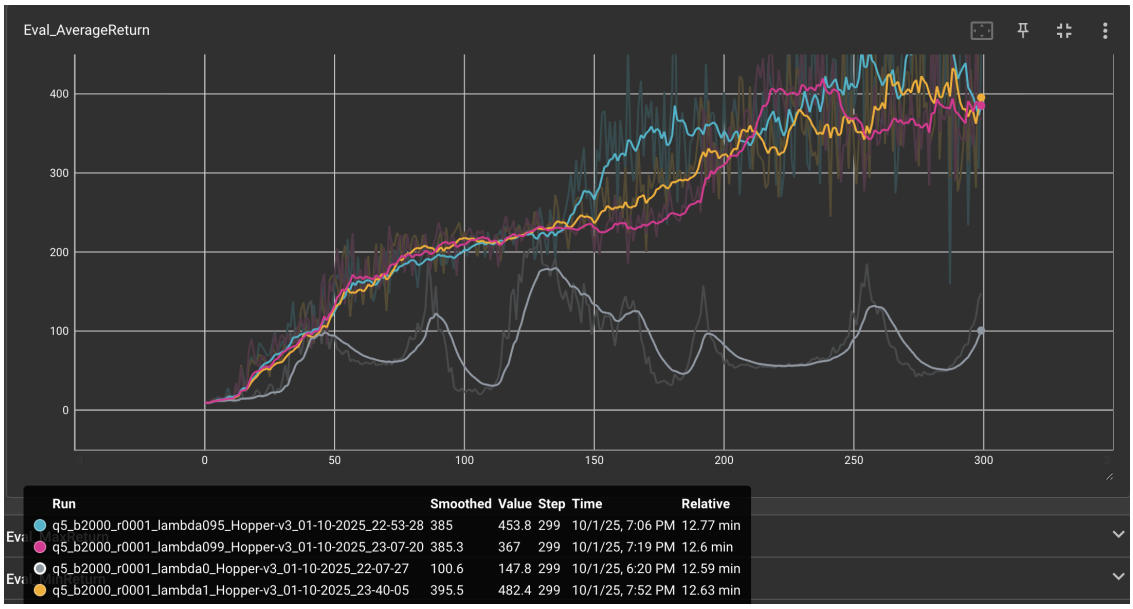
8.1.1 Configurations

Q8.1.1

```
# λ ∈ [0, 0.95, 0.99, 1]
python rob831/scripts/run_hw2.py \
    --env_name Hopper-v4 --ep_len 1000
    --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr 0.001 \
    --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda <λ> \
    --exp_name q5_b2000_r0.001_lambda<λ>
```

8.1.2 Plot – [2 points]

Q8.1.2



8.1.3 Describe how λ affects task performance – [2 points]

Q8.1.3

(I ran the experiment on Hopper-V3 due to colab issues. The graph is smoothed for better visualization) When we set lambda equals to 0, it is a pure TD error. Although the reward curve is much more consistent than the others, it is consistently bad as it is limited by the quality of the nn_baseline.
When we set lambda close to 1, we collect data and rewards from rollouts using Monte Carlo, centered at nn_baseline. In this case, while the final rewards look great, it exhibits high variance as shown in the un-smoothed curves. A single lucky or unlucky rollout can cause the reward to surge or drop significantly.

9 More Bonus!

9.1 Parallelization – [1.5 points]

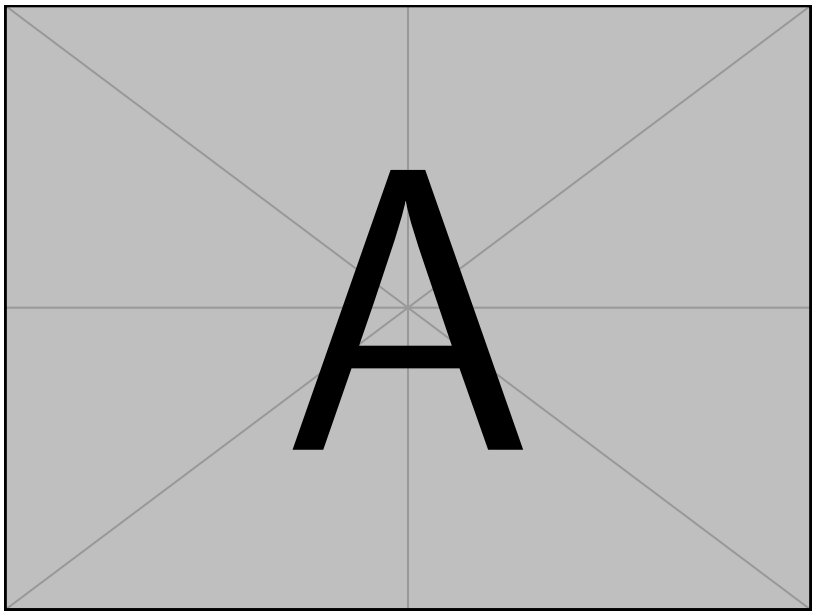
Q9.1

Difference in training time:

```
python rob831/scripts/run_hw2.py \
```

9.2 Multiple gradient steps – [1 points]

Q9.1




```
python rob831/scripts/run_hw2.py \
```